Table of Contents

1. Org-Babel + emacs-jupyter + Python 導入
    1. 処理の流れ
    2. 環境構築
        1. Emacs が動的モジュールサポート付きかどうか確認する
        2. **MELPA** を使えるようにする
        3. `init.el` で `jupyter` を有効にする
        4. `package.el` でパッケージを記述する
        5. 仮想環境を作成する
        6. `config.el` に `jupyter.exe` が入っているフォルダのパスを明記する
        7. 実行の確認
        8. 実行場所の確認
    3. コードハイライト
    4. 参考リンク

Org-Babel + emacs-jupyter + Python 導入

非同期処理と描画埋め込みのできる **emacs-jupyter** を導入します。

- **Org-Babel** : Org-mode でコードブロックを実行する仕組み
- **emacs-jupyter** : その中で Jupyter kernel を使って Python などを実行する拡張

処理の流れ

1. Org-mode で Org-Babel が提供するコードブロックに Python コードを記述する
2. **C-c C-c** で実行する(Org-Babel 経由での実行)
3. emacs-jupyter が **ZeroMQ** という通信を通して **Jupyter Kernel** に送る
4. Jupyter Kernel から出力が返ってくる
5. emacs-jupyter が結果を Org-mode(Org-Babel) に返す

環境構築

Emacs が動的モジュールサポート付きかどうか確認する

Elisp では記述しきれない **ZeroMQ** などの高度な技術を呼び出すために 「動的モジュールサポート」が必要です。

最近の OS 標準パッケージや Homebrew 等でインストールした Emacs であれば概ね有効になっているそうですが、念のため確認します。

1. `M-:` を押し、評価モードに入る
2. ミニバッファに `Eval:` と出るので、以下を入力する。 `t(=True)` であれば動的モジュールサポートが付いている

MELPA を使えるようにする

MELPA は 「Emacs のための非公式パッケージストア」 です。 GNU ELPA という公式のパッケージストアが標準機能としてありますが、 条件が厳しいためほとんどの拡張機能は MELPA にて配布されています。

Doom Emacs では既に有効化されています。

`init.el` で `jupyter` を有効にする

(org +jupyter) に変更します。 Org-Babel は Doom Emacs では既に有効化されているので書く必要はありません。

```
;; init.el
:lang
(org +jupyter)
```

`package.el` でパッケージを記述する

```
;; package.el
(package! jupyter)
(package! simple-httpd :recipe (:host github :repo "skeeto/emacs-web-server"))
(package! emacs-websocket :recipe (:host github :repo "ahyatt/emacs-websocket"))
```

仮想環境を作成する

1. 現在の環境の確認
   ipykernel には Python 3.10 以上が必要です。詳細は ipykernel ドキュメント を参照してください。
   - Windows

     ```
     python --version
     ```

   - WSL（AlmaLinux10）

     ```
     python3 --version
     ```

2. 仮想環境の作成
   先ほど確認したバージョンと、仮想環境を作りたいディレクトリを指定して、作成します。
   - Windows

     ```
     py -3.12 -m venv C:\Users\nomura\myenv312
     ```

     確認

```
C:\Users\nomura\myenv312\Scripts\python.exe --version
```

- WSL（AlmaLinux10）

```
python3.12 -m venv ~/myenv312
```

確認

```
ls -ld ~/myenv312
```

3. 必要なパッケージのインストール
   ipykernel を入れていきます。
   ipykernel は依存関係として `jupyter_client` と `jupyter_core` を一緒に入れてくれます。
   よってインストール時に `jupyter` を指定することなく、jupyter コマンドが使用できます。 万が一 jupyter コマンドが使用できない場合は別途 jupyter を入れてください。 詳細は ipykernel 依存関係 を参照

   - Windows

```
C:\Users\nomura\myenv312\Scripts\python.exe -m pip install ipykernel
```

jupyter コマンドが使用できるかどうかの確認

```
C:\Users\nomura\myenv312\Scripts\jupyter.exe --version
```

使用できない場合は以下のコマンドで jupyter を追加インストール

```
C:\Users\nomura\myenv312\Scripts\python.exe -m pip install jupyter
```

   - WSL（AlmaLinux10） ipykernel インストール

```
~/myenv312/bin/python -m pip install ipykernel
```

jupyter コマンド確認

```
~/myenv312/bin/jupyter --version
```

確認できない場合は別途インストール

```
~/myenv312/bin/python -m pip install jupyter
```

4. 仮想環境を jupyter のカーネルとして登録する ipykernel を使って仮想環境を jupyter のカーネルとして登録します。
   登録時にいくつか指定するべき項目があります。
   詳細は docs/kernels.rst または IPython kernel ドキュメントを参照してください。

> `--user` : Windows のユーザー単位で登録
> `--name` : カーネルの（内部の）名前（任意。Org-Babel のコードブロックにカーネルを指定するときに使う名前です。）
> `--display-name` : UI に表示されるカーネル名

   - Windows

```
C:\Users\nomura\myenv312\Scripts\python.exe -m ipykernel install --user
--name emacs-venv --display-name "Python (Emacs Venv)"
```

登録済みカーネルの確認

```
C:\Users\nomura\myenv312\Scripts\jupyter.exe kernelspec list
```

   - WSL（AlmaLinux10）

```
~/myenv312/bin/python -m ipykernel install --user --name emacs-venv --
display-name "Python (Emacs Venv)"
```

登録済みカーネルの確認

```
~/myenv312/bin/jupyter kernelspec list
```

`config.el` に `jupyter.exe` が入っているフォルダのパスを明記する

作成した仮想環境のパスを `config.el` に記述します。

```
;; config.el
(after! jupyter
    (setq jupyter-executable "C:/Users/light/myenv312/Scripts/jupyter.exe")
    (add-to-list 'exec-path "C:/Users/light/myenv312/Scripts"))
```

emacs-jupyter ドキュメント Org Babel source block を参照して、Org-Babel の言語に jupyter を登録します。

また、emacs-jupyter のデフォルトは同期実行です。コードブロックで `:async yes` とすれば非同期実行になります。
デフォルトを非同期実行にしてほしいので設定に書き加えます。同期実行のままでよい場合は飛ばしてください。

```
;; config.el
(after! org
    (require 'jupyter)

    ;; jupyter を org-babel-load-languages に追加する
    (org-babel-do-load-languages
    'org-babel-load-languages
    '((python . t)
        (jupyter . t)))

    ;; デフォルト設定を非同期実行に変更
    (setq org-babel-default-header-args:jupyter-python
        '((:async . "yes")))
    (setq org-babel-default-header-args:python
        '((:async . "yes")))
```

実行の確認

グラフ描画のコードを実行してみます。
numpy と matplotlib を入れます。

- Windows

  ```
  C:\Users\nomura\myenv312\Scripts\python.exe -m pip install -U numpy
  matplotlib
  ```

- WSL（AlmaLinux-10）

  ```
  ~/myenv312/bin/python -m pip install numpy matplotlib
  ```

Org ファイルにコードブロックを記述します。
セッション名(:session)は自由に決めてください。

カーネル名(:kernel)は仮想環境を jupyter のカーネルとして登録する際に指定した `--name` を使用します。

```
#+BEGIN_SRC jupyter-python :session py :kernel emacs-venv
;; コード
#+END_SRC
```

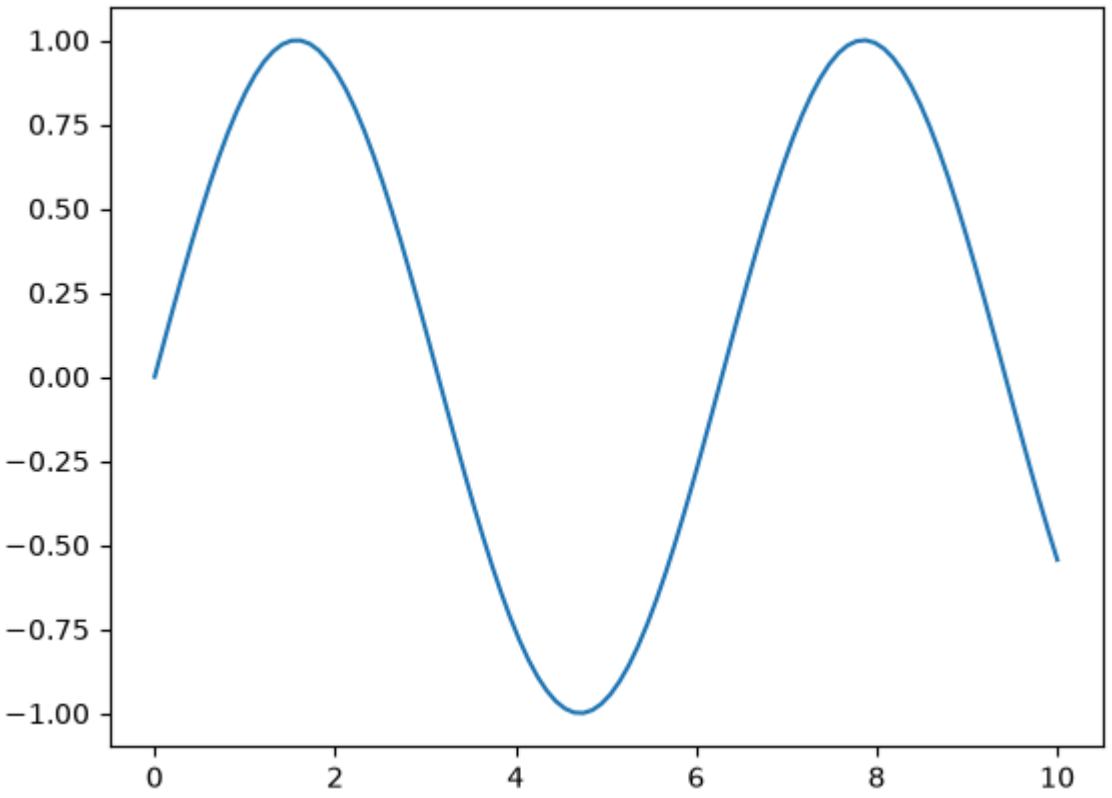
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
plt.plot(x, np.sin(x))
plt.show()

import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
plt.plot(x, np.sin(x))
plt.show()
```

以下のような sin カーブが出力されれば成功です。



実行場所の確認

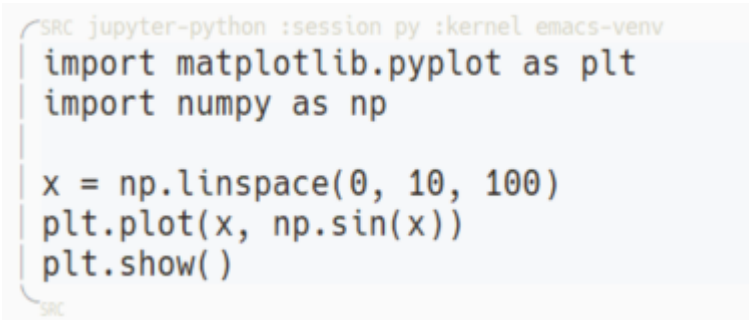
Python コード側から、実行場所を見てみます。
作成した仮想環境のパスが出力されれば成功です。

```
import sys
print(sys.executable)
```

コードハイライト

Org ファイルのコードハイライトは `org-src-lang-modes` という対応表を見て行われます。
初期状態では jupyter-python と python の対応 `("jupyter-pythom" . python)` が対応表にないため、コードに色がつきません。（実行すればつきますが、ファイルを開いた時点でハイライトしてほしいです）

↓ ファイルを開いた段階ではハイライトはつかない



`config.el` に (“jupyter-python” . python) を明記します。
これで jupyter-python と python の対応ができるのでコードがハイライトされます。

```
;; config.el
(after! org
    (setq org-src-fontify-natively t)
    ;; jupyter言語名 -> major mode の対応を追加
    (add-to-list 'org-src-lang-modes '("jupyter-python" . python)))
```

参考リンク

ipykernel ドキュメント https://pypi.org/project/ipykernel/
ipykernel 依存関係 https://github.com/ipython/ipykernel/blob/main/pyproject.toml
docs/kernels.rst https://github.com/jupyter/jupyter_client/blob/main/docs/kernels.rst
IPython kernel ドキュメント https://ipython.readthedocs.io/en/latest/install/kernel_install.html
emacs-jupyter ドキュメント Org Babel source block https://github.com/emacs-jupyter/jupyter#org-mode-source-blocks